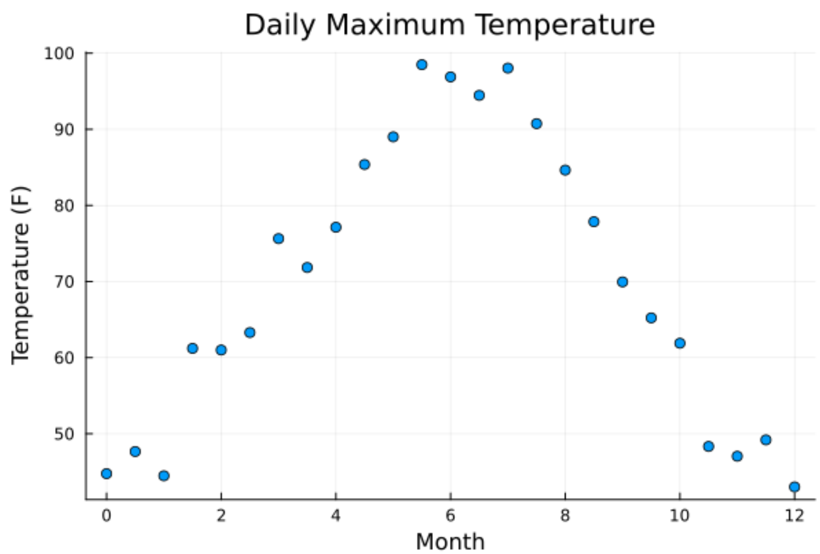
EE263 Homework 6

Fall 2025

due Thursday 11/6, at 11:59 PM

6.3200. Fitting a Piecewise Linear Function to Data. Last year, we sampled the maximum daily temperature outside Packard twice a month. Looking at the plotted results, we believe there's a clear trend in the temperatures over the time of year. We have collected $n+1=25$ datapoints. Each data point consists of two values: x and y. The x value ranges from 0 to 12 and describe when the data was collected in months since the start of the year. The y value is the recorded temperature in Fahrenheit. We have two data sets, a training set and a test set, in the file `tempdata.json`.



In order to better describe the relationship between time of year and daily maximum temperature, we will fit a piecewise linear function g to the training data set. We will use $m+1$ piecewise affine functions $f_0, f_1, \ldots, f_m$, and approximate the data by the function g, which is a linear combination of them.

$$g(x) = \sum_{j=0}^{m} \alpha_j f_j(x)$$

Here $f_0 = 1$ and for $j = 1, \ldots, m$ we have

$$f_j(x) = \begin{cases} 0 & \text{if } x < (j-1)\frac{12}{m} \\ \frac{mx}{12} - (j-1) & \text{if } (j-1)\frac{12}{m} \le x \le j\frac{12}{m} \\ 1 & \text{if } j\frac{12}{m} < x \end{cases}.$$

The functions f_i are very simple piecewise linear step functions. We recommend graphing a couple for variable m and j to get intuition for what these functions are.

a) Our objective is to select weights $\alpha_0, \ldots, \alpha_m$ to minimize

$$\sum_{i=0}^{n} \|y_i - g(x_i)\|_2^2.$$

Express this objective in the form

$$\text{minimize } \|y - F\alpha\|_2^2$$

for some known vector y, known matrix F, and unknown vector α.

b) Suppose n is a multiple of m, and there are $n+1$ data points that are evenly spaced in x from 0 to 12 inclusive, so that gap between points is $12/n$. Show that the matrix F is full rank.

c) Use Julia to solve this problem for $m = 3, 6, 12, 24$ for the training data x^{train} and y^{train}. Plot the data points (x, y) along with the fitted function g for each value of m.

d) Plot the minimal squared 2-norm error ($\|y - F\alpha\|_2^2$) for $m = 3, 6, 12, 24$. Is there a point where adding more complexity to the model (increasing m) offers clearly diminishing returns?

e) We now turn to validation of the fit. We have an additional data set, x^{test} and y^{test}, which we will use to test the accuracy of our model. The test error is

$$J^{\text{test}} = \sum_{i=0}^{n_{\text{test}}} \|y_i^{\text{test}} - g(x_i^{\text{test}})\|_2^2.$$

Here g is the function you found in part (c) above. Plot the test error J^{test} versus m for $m = 3, 6, 12, 24$. Note that this does not involve recomputing α. What does this say about your answer to part (d).

7.1040. Fitting a Gaussian function to data. A Gaussian function has the form

$$f(t) = ae^{-(t-\mu)^2/\sigma^2}.$$

Here $t \in \mathbb{R}$ is the independent variable, and $a \in \mathbb{R}$, $\mu \in \mathbb{R}$, and $\sigma \in \mathbb{R}$ are parameters that affect its shape. The parameter a is called the *amplitude* of the Gaussian, μ is called its *center*, and σ is called the *spread* or *width*. We can always take $\sigma > 0$. For convenience we define $p \in \mathbb{R}^3$ as the vector of the parameters, *i.e.*, $p = [a\ \mu\ \sigma]^\mathsf{T}$. We are given a set of data,

$$t_1, \ldots, t_N, \qquad y_1, \ldots, y_N,$$

and our goal is to fit a Gaussian function to the data. We will measure the quality of the fit by the root-mean-square (RMS) fitting error, given by

$$E = \left(\frac{1}{N} \sum_{i=1}^{N} (f(t_i) - y_i)^2 \right)^{1/2}.$$

Note that E is a function of the parameters a, μ, σ, *i.e.*, p. Your job is to choose these parameters to minimize E. You'll use the Gauss-Newton method.

a) Work out the details of the Gauss-Newton method for this fitting problem. Explicitly describe the Gauss-Newton steps, including the matrices and vectors that come up. You can use the notation $\Delta p^{(k)} = [\Delta a^{(k)} \ \Delta \mu^{(k)} \ \Delta \sigma^{(k)}]^\mathsf{T}$ to denote the update to the parameters, *i.e.*,

$$p^{(k+1)} = p^{(k)} + \Delta p^{(k)}.$$

(Here k denotes the kth iteration.)

b) Get the data t, y (and N) from the file `gauss_fit_data.json`, available on the class website. Implement the Gauss-Newton (as outlined in part (a) above). You'll need an initial guess for the parameters. You can visually estimate them (giving a short justification), or estimate them by any other method (but you must explain your method). Plot the RMS error E as a function of the iteration number. (You should plot enough iterations to convince yourself that the algorithm has nearly converged.) Plot the final Gaussian function obtained along with the data on the same plot. Repeat for another reasonable, but different initial guess for the parameters. Repeat for another set of parameters that is *not* reasonable, *i.e.*, not a good guess for the parameters. (It's possible, of course, that the Gauss-Newton algorithm doesn't converge, or fails at some step; if this occurs, say so.) Briefly comment on the results you obtain in the three cases.

8.1460. Filling-in missing data. In this problem we have a signal, $y_i \in \mathbb{R}$ for $i = 1, \ldots, n$, which we view as $y \in \mathbb{R}^n$. We will have $n = 100$. The signal y comes from measurements of a physical system, and so y_{i+1} is measured a short time interval after y_i. Unfortunately, during the data acquisition process some of the data was lost and so the signal we have has gaps in it. Specifically, we have a known set $K \subset \mathbb{Z}$ and we know y_i only for values $i \in K$.

The data for this problem is in the file `missing.json`. The supplied vector `known` contains the list of known points K, and the vector `yknown` is the list of values of y at the points in K. The length of `yknown` is therefore $|K|$.

a) For a signal $z \in \mathbb{R}^n$, we define the discrete derivative $z^{\text{der}} \in \mathbb{R}^{n-1}$ by

$$z_i^{\text{der}} = z_{i+1} - z_i \quad \text{for } i = 1, \ldots, n-1$$

Find the matrix G such that $z^{\text{der}} = Gz$

b) Our first approach will be to find the signal z which minimizes $\|z^{\text{der}}\|$ and satisfies

$$z_i = y_i \quad \text{if } i \in K$$

Give a method finding the optimal z.

c) Find the optimal z in the previous part and plot z_i against i. Be sure to plot the points (i, z_i), not just a line joining them.

d) One way to do a better job at filling in the missing data is to put additional criteria on our estimate. Here we will do this by additionally penalizing the second derivative of z. Define the discrete second derivative $z^{\text{hes}} \in \mathbb{R}^{n-2}$ by

$$z_i^{\text{hes}} = z_{i+2} - 2z_{i+1} + z_i \quad \text{for } i = 1, \dots, n-2$$

Find the matrix H such that $z^{\text{hes}} = Hz$

e) Define the two objective functions

$$J_1 = \|Gz\|^2 \qquad J_2 = \|Hz\|^2$$

We would like to find the signal z that minimizes

$$J_1 + \mu J_2$$

and satisfies

$$z_i = y_i \quad \text{if } i \in K$$

Give a method for finding the optimal z.

f) Plot the trade-off curve of J_2 (on the vertical axis) versus J_1 (on the horizontal axis). Give the interpretation of the endpoints of this curve.

g) Find the optimal z for the three different cases $\mu = 5, 20, 100$.

11.1830. Estimating a matrix with known eigenvectors. This problem is about estimating a matrix $A \in \mathbb{R}^{n \times n}$. The matrix A is not known, but we do have a noisy measurement of it, $A^{\text{meas}} = A + E$. Here the matrix E is measurement error, which is assumed to be small. While A is not known, we do know real, independent eigenvectors $v_1, \dots, v_n$ of A. (Its eigenvalues $\lambda_1, \dots, \lambda_n$, however, are not known.) We will combine our measurement of A with our prior knowledge to find an estimate $\hat{A}$ of A. To do this, we choose $\hat{A}$ as the matrix that minimizes

$$J = \frac{1}{n^2} \sum_{i,j=1}^{n} (A_{ij}^{\text{meas}} - \hat{A}_{ij})^2$$

among all matrices which have eigenvectors $v_1, \dots, v_n$. (Thus, $\hat{A}$ is the matrix closest to our measurement, in the mean-square sense, that is consistent with the known eigenvectors.)

a) Explain how you would find $\hat{A}$. If your method is iterative, say whether you can guarantee convergence. Be sure to say whether your method finds the exact minimizer of J (except, of course, for numerical error due to roundoff), or an approximate solution. You can use any of the methods (least-squares, least-norm, Gauss-Newton, low rank approximation, *etc.*) or decompositions (QR, SVD, eigenvalue decomposition, *etc.*) from the course.

b) Carry out your method with the data

$$A^{\text{meas}} = \begin{bmatrix} 2.0 & 1.2 & -1.0 \\ 0.4 & 2.0 & -0.5 \\ -0.5 & 0.9 & 1.0 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 0.7 \\ 0 \\ 0.7 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0.3 \\ 0.6 \\ 0.7 \end{bmatrix}, \quad v_3 = \begin{bmatrix} 0.6 \\ 0.6 \\ 0.3 \end{bmatrix}.$$

Be sure to check that $\hat{A}$ does indeed have v_1, v_2, v_3 as eigenvectors, by (numerically) finding its eigenvectors and eigenvalues. Also, give the value of J for $\hat{A}$. **Hint.** You might find the following useful (but then again, you might not.) In Julia, if `A` is a matrix, then `A[:]` is a (column) vector consisting of all the entries of `A`, written out column by column. Therefore `norm(A[:])` gives the squareroot of the sum of the squares of entries of the matrix `A`, *i.e.*, its Frobenius norm. The inverse operation, *i.e.*, writing a vector out as a matrix with some given dimensions, is done using the function `reshape`. For example, if `a` is an mn vector, then `reshape(a,m,n)` is an $m \times n$ matrix, with elements taken from `a` (column by column).

15.2200. Properties of symmetric matrices. In this problem P and Q are symmetric matrices. For each statement below, either give a proof or a specific counterexample.

a) If $P \geq 0$ then $P + Q \geq Q$.

b) If $P \geq Q$ then $-P \leq -Q$.

c) If $P > 0$ then $P^{-1} > 0$.

d) If $P \geq Q > 0$ then $P^{-1} \leq Q^{-1}$.

e) If $P \geq Q$ then $P^2 \geq Q^2$.

Hint: you might find it useful for part (d) to prove $Z \geq I$ implies $Z^{-1} \leq I$.